# How the MCP Server Is Built and How It Works

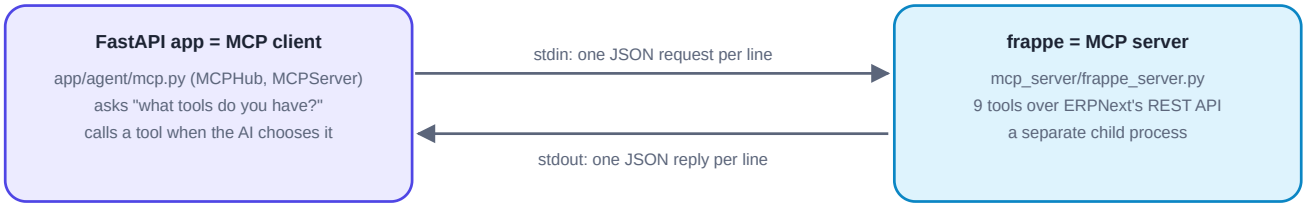
A guided tour of the **frappe** MCP server in the AI-Assisted Activity Tracker: what was built, why each piece exists, and what happens to a request from Telegram to ERPNext.

## 1. What MCP is, in plain words

**MCP (Model Context Protocol)** is a small standard that lets an AI application plug in extra tools. There are two roles:

- the **client**: our FastAPI app, which wants to offer more tools to the AI;
- the **server**: a separate small program that owns those tools. Ours is `frappe`, and it knows how to read and change ERPNext records.

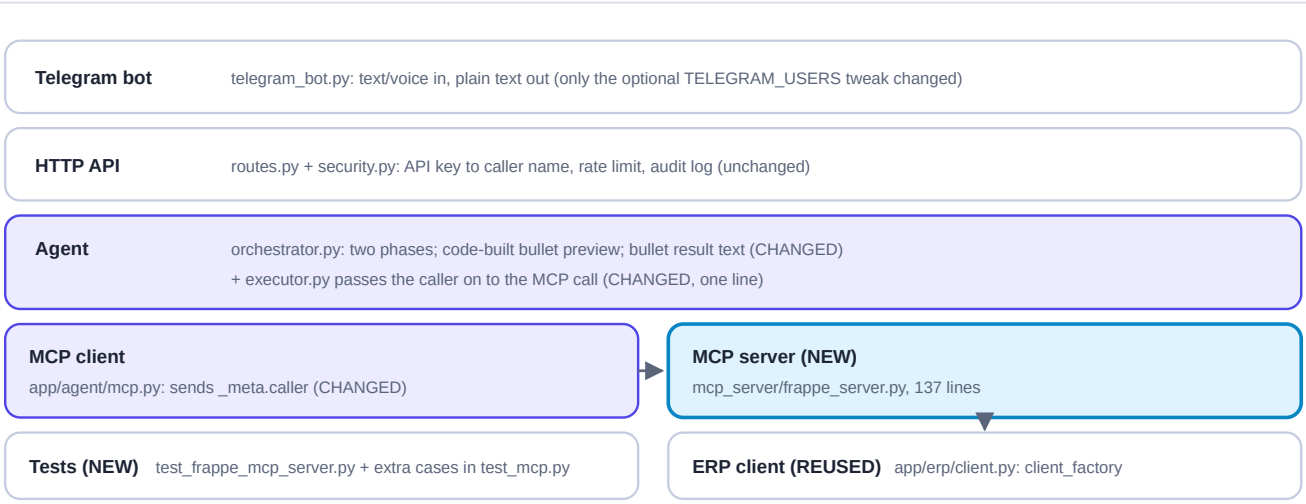
They talk by sending one JSON message per line over the server's standard input and output (**stdio**). No network port, no login: the client simply starts the server as a child process.




The client launches the server itself, so there is nothing to deploy or secure separately.

## 2. What was built, and what already existed

The client half (`mcp.py`) and the two-phase fallback in the orchestrator already existed for third-party servers. This work added our own server and taught the client to say *who* is calling.




Purple = touched or new on the agent/client side. Blue = the new server. The server reuses the existing ERP client, so it adds no new setting and no new dependency.

| FILE | ROLE | STATUS |
|---|---|---|
| `mcp_server/frappe_server.py` | The MCP server: 9 tools, guards, per-caller identity | new (137 lines) |
| `app/agent/mcp.py` | Client: starts servers, lists tools, calls them; now forwards the caller | edited |
| `app/agent/orchestrator.py` | Phase 1/2 logic; bullet preview and result text; curly-apostrophe fix | edited |
| `app/services/executor.py` | Runs a confirmed action; passes `action.caller` | edited |
| `tests/test_frappe_mcp_server.py` | Server behaviour against an in-memory ERPNext stub | new (92 lines) |

## 3. How the server was built, step by step

### Step 1: speak the protocol

The server reads one JSON line at a time and answers three kinds of request: `initialize` (say hello), `tools/list` (what can you do?) and `tools/call` (do this). Notifications (messages without an id) get no reply. It is hand-written in a few lines, the same dialect the client and the test fake already use, so no SDK is needed.

### Step 2: describe the tools

Each tool has a name, a description for the AI, an input schema, and **annotations** saying whether it only reads. That label is what lets the app run reads at once and hold writes for a yes.

```
{"name": "create_doc", "annotations": {"readOnlyHint": false},
 "inputSchema": {"type": "object", "required": ["doctype", "data"], ...}}
{"name": "cancel_doc", "annotations": {"readOnlyHint": false, "destructiveHint": true}, ...}
```

### Step 3: act as the caller

The server calls `client_factory(Settings.from_env())`, the same helper the app uses, which returns an ERPNext client holding **that person's own API key**. The app puts the name in `params._meta.caller`; the server looks the key up from `ERP_USERS`. No caller, no call.

### Step 4: add the guards

Security DocTypes are blocked, writes to User are blocked, Sales Order submit/cancel needs `CONFIRM_CALLERS`, item creation needs `ITEM_CREATORS`, list size is capped by `MAX_PAGE_SIZE`. There is no delete tool and no free-form method call.

### Step 5: wire it in

One line in `.env` turns it on; nothing else is configured:

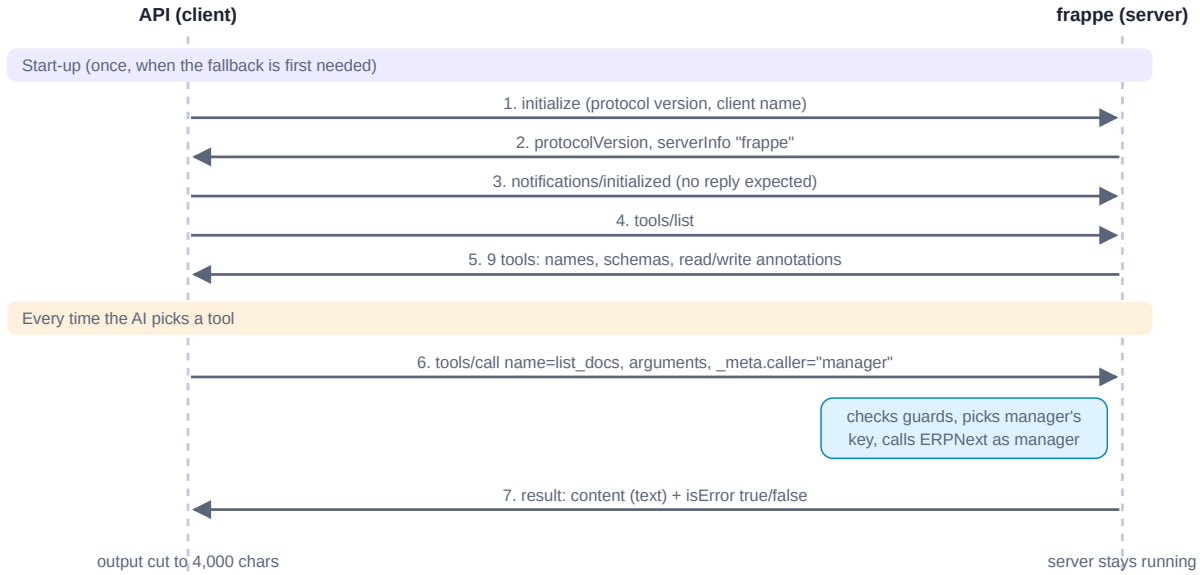
```
MCP_SERVERS=frappe=venv/bin/python -m mcp_server.frappe_server
```

On startup the API builds an `MCPHub` from this setting. The server is started lazily the first time a fallback is needed, restarted if it exits, and stopped when the API shuts down.

### Step 6: test it

Unit tests drive the server directly against an in-memory ERPNext stub; client tests run a real child process (a fake server that logs every call) to prove a write never reaches the server before yes; a smoke test starts the real server and lists its tools; finally the whole flow was run through Telegram.

## 4. The conversation between client and server

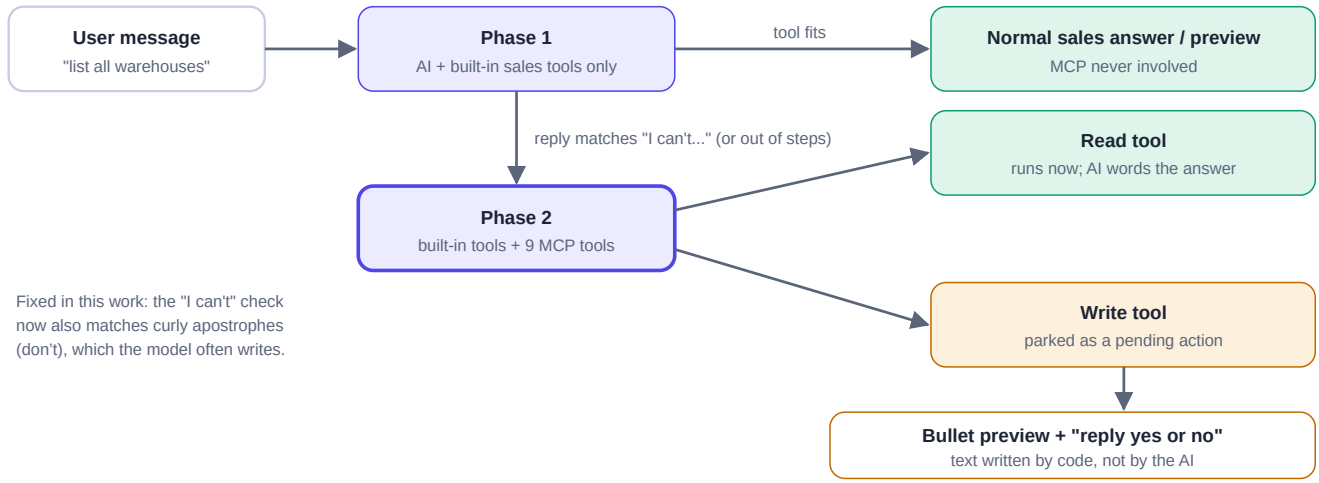



Steps 1 to 5 happen once. Step 6 and 7 repeat for every tool the AI uses. A hung server is dropped after MCP_TIMEOUT (60 s).

What a real call looks like on the wire (one line each in practice):

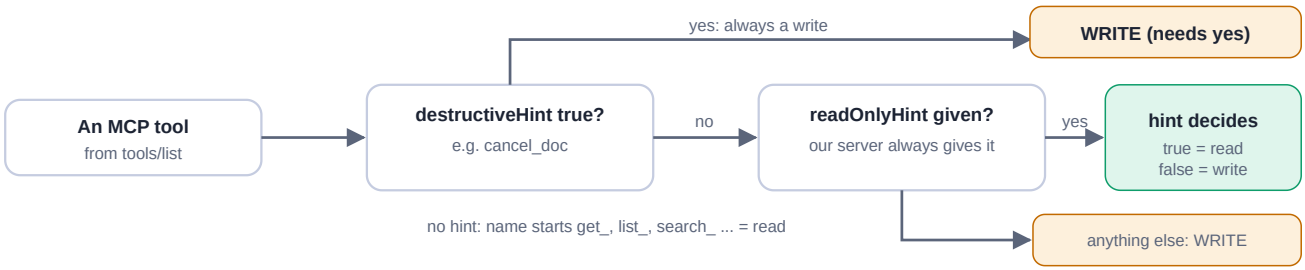
```
-> {"jsonrpc":"2.0","id":7,"method":"tools/call","params":{"name":"list_docs",
    "arguments":{"doctype":"Warehouse","fields":["name"]},"_meta":{"caller":"manager"}}}
<- {"jsonrpc":"2.0","id":7,"result":{"content":[{"type":"text","text":"[{\"name\": \"Stores -
AMT\"}, ...]"}],"isError":false}}
```

# 5. From a Telegram message to an answer



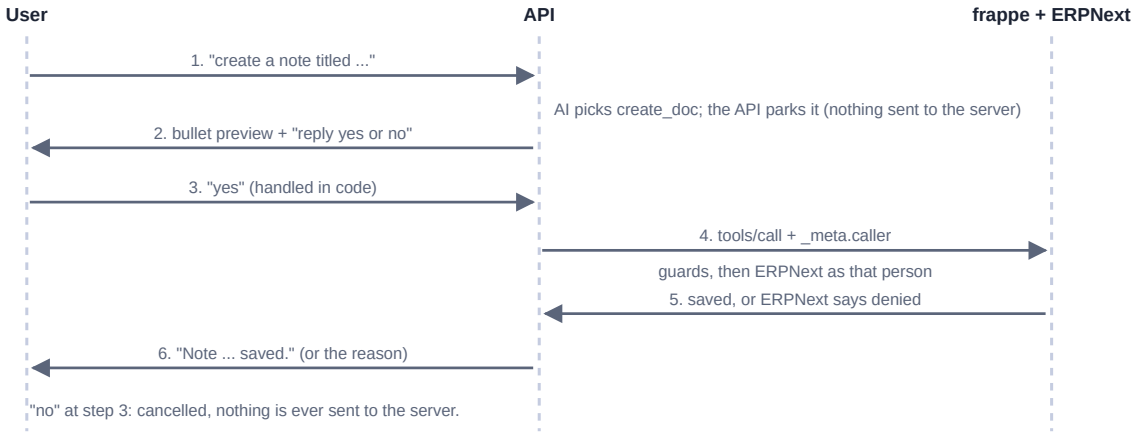

The built-in sales tools always get the first try, so the checked sales flow is never bypassed.

## How a tool is judged "read" or "write"



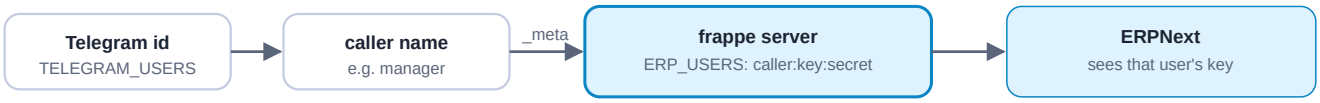

Unknown tools fall on the safe side: if nothing says "read", the user must say yes.

## 6. The yes gate for changes




The action belongs to the person who asked and expires after ACTION_TTL_SECONDS. A newer proposal replaces an older unanswered one.

## 7. Who is calling: per-person identity

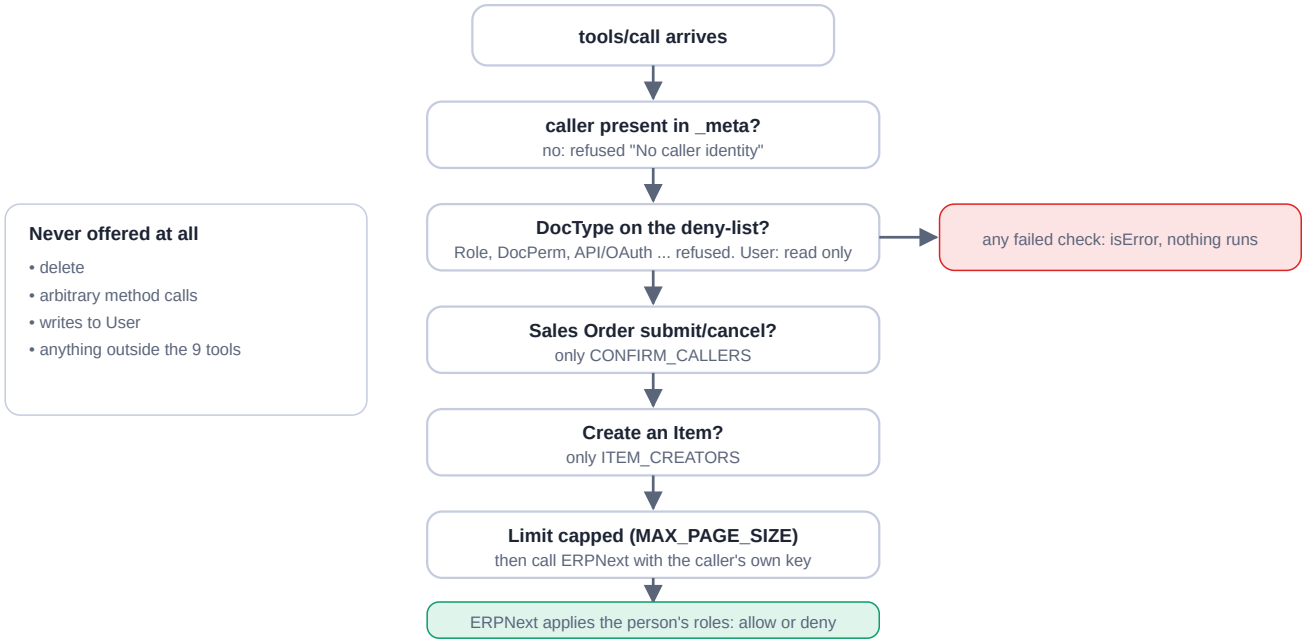



**What this means**

- ERPNext decides allow or deny using that person's own roles (manager was refused on Employee and Customer Group).
- The assistant can never see or change more than the person could in ERPNext.
- A call without a caller is refused; there is no shared admin fallback.
- Extra app-level rules on top: CONFIRM_CALLERS for Sales Order submit/cancel, ITEM_CREATORS for new items.

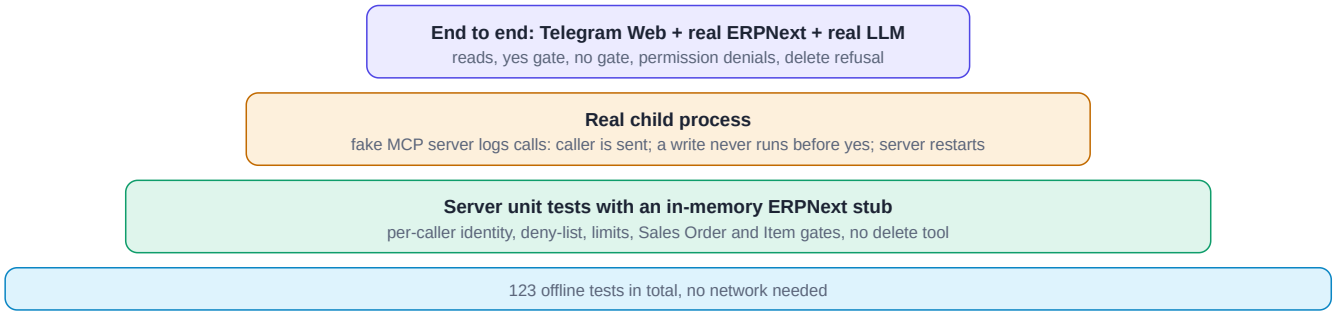
The three config maps (TELEGRAM_USERS, ERP_USERS, API_KEYS) must name the same callers.

## 8. Inside the server: the guards on every call




All failures return a readable message with isError true, which the AI turns into a short reply.

## 9. How it was tested




Each layer proves something the layer below cannot: the stub proves the rules, the child process proves the protocol and the yes gate, Telegram proves the whole chain.

## 10. Adding another tool later

- Add one entry to `TOOLS` in `frappe_server.py` with the right `readOnlyHint`.
- Add a branch in `Server.call`, and a guard in `_check` if it is sensitive.
- Add a case to `tests/test_frappe_mcp_server.py`.
- No change in the app is needed: the client discovers new tools with `tools/list` and treats them as read or write from the annotation.

Limits to remember: the MCP tools are only offered after the built-in tools decline; the built-in fuzzy-name and price rules do not apply to MCP requests; pending confirmations live in memory and are lost on restart.